Python OOP Foundations

Mastering Scalable Structure in the AI Era

> *"If you want to drive the AI, you must understand the structure. Production-level code is 99.9% written in Classes."*

1. The Core Problem: Scalability

Imagine you need to manage details for 200,000 employees (First Name, Last Name, Pay, Email).

The Wrong Way: Hardcoding

Writing individual variables for every person is unsustainable and prone to errors.

```
# Hardcoding - Not sustainable!
emp_1_first = "Rahul"
emp_1_last = "Sharma"
emp_1_pay = 50000

emp_2_first = "Priya"
emp_2_last = "Patel"
emp_2_pay = 40000
```

The Better Way: Functions

Functions help, but they create a "disconnection" between the data (employee info) and the behavior (logic for raises or formatting).

```
def create_employee(first, last, pay):
    email = f"{first.lower()}.{last.lower()}@infosys.com"
    return {"first": first, "last": last, "email": email, "pay": pay}

emp_1 = create_employee("Rahul", "Sharma", 50000)
```

2. What is a Class?

A **Class** is a blueprint for creating objects. Think of it like a house blueprint: The blueprint isn't a house, but it tells you exactly how to build one.

- **Object (Instance):** An actual house built from that blueprint.

3. Creating Your First Class: __init__ and self

The __init__ Method

This is a special "Dunder" (Double underscore) method that runs automatically every time you create a new object. It initializes the data.

The self Parameter

`self` is a placeholder for the specific instance being created. When you create `emp_1`, `self` becomes `emp_1`.

```
class Employee:
    # The constructor/initializer
    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.pay = pay
        # Deriving data inside the init
        self.email = f"{first.lower()}.{last.lower()}@infosys.com"

# Creating instances (Objects)
emp_1 = Employee("Rahul", "Sharma", 50000)
emp_2 = Employee("Priya", "Patel", 40000)
```

4. Instance Variables

Instance variables are unique to each object. Changing one does not affect the other.

Analogy: Changing the address on your ID card doesn't change it on everyone else's card.

```
# Updating pay only for emp_1
emp_1.pay = 850000

print(emp_1.pay) # 850000
print(emp_2.pay) # 40000 (remains unchanged)
```

Pro Tip: Use `__dict__` to see all variables held by an instance: `print(emp_1.__dict__)`

5. Adding Methods (Behaviors)

A **Method** is simply a function defined inside a class. It must always take `self` as the first argument to access the object's data.

```
class Employee:
    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.pay = pay

    def full_name(self):
        return f"{self.first} {self.last}"

    def increase_pay(self, percentage):
        self.pay = int(self.pay * (1 + percentage / 100))

emp_1 = Employee("Rahul", "Sharma", 800000)
print(emp_1.full_name()) # Output: Rahul Sharma
emp_1.increase_pay(4)
print(emp_1.pay) # Output: 832000
```

6. Two Ways to Call Methods

- **Instance Call:** `emp_1.full_name()`
- **Class Call:** `Employee.full_name(emp_1)` (Requires manually passing the instance).

Production Level Code Checklist:

- Use **CamelCase** for Class names (e.g., `EmployeeData`).
- Always use `self` as the first argument in methods.
- Encapsulate data and logic together to drive AI tools effectively.

"Production-level code is 99.9% written in Classes."
